Optimization-Aware Test Generation for Deep Learning Compilers

Qingchao Shen[1], Zan Wang[1], Haoyang Ma[2], Yongqiang Tian[3], Lili Huang[1], Zibo Xiao[1]
Junjie Chen[1,*], Shing-Chi Cheung[2]
[1]School of Computer Software, Tianjin University, Tianjin, China
Email: {qingchao, wangzan, huangll, zibo.xiao, junjiechen}@tju.edu.cn
[2]The Hong Kong University of Science and Technology, Hong Kong, China
Email: haoyang.ma@connect.ust.hk, scc@cse.ust.hk
[3]Monash University, Melbourne, Australia, Email: yongqiang.tian@monash.edu

arXiv:2511.18918v1 [cs.SE] 24 Nov 2025

Abstract

Deep Learning (DL) compilers have been widely utilized to optimize DL models for efficient deployment across various hardware. Due to their vital role in the DL ecosystem, ensuring their reliability is critical. Such model optimizations are often designed to match specific computational graph structures. However, existing DL compiler fuzzing techniques do not generate tests for each optimization aware of its matched graph structures. In this paper, we propose OATest, a novel technique that synthesizes optimization-aware tests by extracting patterns from documented tests and integrating them into diverse computational graph contexts. To address the key technical challenge of synthesizing valid and optimization-aware computational graphs, OATest introduces two synthesis strategies: (1) reusing compatible inputs/outputs from existing nodes and (2) creating new nodes with compatible inputs/outputs, establishing effective connections between patterns and contexts. OATest is evaluated on two popular DL compilers, TVM and ONNXRuntime, regarding the bugs revealed by crashes and inconsistencies. The experimental results show that OATest significantly outperforms the state-of-the-art DL compiler fuzzing techniques by detecting more bugs and covering more optimization code in TVM and ONNXRuntime. Particularly, OATest uncovers 56 previously unknown bugs, 42/24 of which have been confirmed/fixed by developers.

Keywords

Compiler Testing, Fuzzing, Test Synthesis, Deep Learning Compiler

ACM Reference Format:
Qingchao Shen[1], Zan Wang[1], Haoyang Ma[2], Yongqiang Tian[3], Lili Huang[1], Zibo Xiao[1], Junjie Chen[1,*], Shing-Chi Cheung[2] . 2026. Optimization-Aware Test Generation for Deep Learning Compilers. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3773216

1 Introduction

Deep Learning (DL) compilers (*e.g.*, TVM [19], ONNXRuntime [39], and TensorRT [2]) have become increasingly popular and play a fundamental role in optimizing DL models for efficient deployment on diverse hardware platforms. However, DL compilers are also vulnerable to bugs, in which compiler optimization (the key component) is more error-prone than other components [42]. Moreover, due to the fundamental role of DL compiler optimizations, their bugs could produce serious consequences [59], causing compiled models to execute incorrectly (leading to unintended behavior) , or even causing runtime failures on specific hardware (leading to service unavailability). Therefore, it is imperative to ensure the quality of DL compiler optimizations.

In recent years, several DL compiler fuzzing techniques [34, 37, 54] have been proposed. The most widely-studied ones are grammar-based techniques, such as NNSmith [34] and HirGen [37]. They focus on generating valid tests, *i.e.*, computational graphs (with nodes representing operators and edges signifying data dependencies between operators), based on grammar for fuzzing DL compilers. However, triggering an optimization tends to require specific patterns [54], *e.g.*, only the structure of `Concat(PermuteDims(A), PermuteDims(B))`, which first rearranges the dimensions of tensors `A` and `B` using `PermuteDims` and then concatenates them into a single tensor, can be optimized by the `ReorderPermuteDimsAfterConcat` optimization (also called pass in the field of compilers) and thus may trigger potential bugs in this optimization. Such grammar-based techniques just arbitrarily construct tests based on grammar without special attention on diverse optimizations, leading to limited effectiveness in detecting optimization bugs. For example, NNSmith covers only 24.75% of TVM's optimization code branches during 12-hours fuzzing (to be presented in Section 4.3).

Unlike grammar-based techniques, WhiteFox [54] was introduced recently, which leverages Large Language Models (LLMs) to mine optimization patterns from compiler source code and generate tests based on them. While WhiteFox has successfully uncovered a certain number of optimization bugs, it still has two main limitations. First, WhiteFox relies on the capability of LLMs on code comprehension, but the compiler source code is much more complicated, limiting its effectiveness in extracting correct patterns and thus generating optimization-aware tests. For example, the `DataflowInsertInPlaceCalls` pass requires an in-place operator, but the pattern extracted by WhiteFox (*i.e.*, `t2 = t1 + t1`) is creating a new tensor (*i.e.*, `t2`) rather than modifying `t1` in place, which does not align with this optimization. Second, LLMs' hallucinations limit the effectiveness of generating valid tests, thereby limiting the overall fuzzing effectiveness. For instance, more than half of the tests generated by WhiteFox are invalid (as detailed in Section 4.5) and have to be rejected before any optimizations are performed. This motivates more research efforts to bridge the gap in designing effective test generation techniques for DL compiler optimizations.

To improve the effectiveness in detecting DL compiler optimization bugs, the generated tests should be as optimization-aware and

*Junjie Chen is the corresponding author.

diverse as possible. To enhance the possibility of triggering DL compiler optimizations, we choose a more straightforward source, *i.e.*, documented tests for each optimization, to learn optimization-aware patterns. On the one hand, these tests are manually written by developers to test the corresponding optimizations and thus embody the specific optimization-aware patterns. On the other hand, such patterns can be extracted through computational graph analysis according to the optimization characteristics, which is a more deterministic manner and thus avoids the non-determinism and inaccuracy incurred by LLMs in comprehending complicated compiler source code. To increase test diversity, we synthesize tests by integrating a pattern corresponding to an optimization into different contexts. Given the context-sensitive nature of DL compiler optimizations [19], this integration may trigger different optimization behaviors, indicating the construction of diverse optimization scenarios to some degree.

Based on these insights, we propose our idea of computational graph synthesis by integrating the optimization-aware patterns learned from documented tests into diverse contexts for improving the effectiveness of fuzzing DL compiler optimizations. Specifically, OATest contains three main steps. First, OATest extracts optimization-aware patterns from the computational graphs gathered from documented tests via code instrumentation. Here, different optimizations work at different granularities (*i.e.*, block - a self-contained sequence of operations, and subgraph - a connected portion of a computation graph) [31], and thus OATest extracts block-level or subgraph-level patterns accordingly in order to ensure the optimization awareness of the extracted patterns. Second, OATest synthesizes tests by integrating extracted patterns into different contexts provided by a set of existing tests. Simply putting them together will lead to invalid tests (*i.e.*, undefined inputs for the operators in patterns), OATest designs two strategies to fix such problems, which is also its key technical challenge. To establish effective connections between patterns and contexts, one fixing strategy in OATest is to reuse the compatible inputs/outputs of the existing nodes in the context for creating edges to the nodes with undefined inputs in the pattern. This is helpful to enhance the interaction between patterns and contexts for activating new optimization behaviors. As it is hard to ensure the existence of compatible inputs/outputs in the context, another fixing strategy is complementary, which creates new nodes with compatible inputs/outputs and then connects them to the nodes with undefined inputs in the pattern. Particularly, OATest enables iterative synthesis of patterns across diverse contexts, facilitating the combination of multiple patterns for fuzzing DL compiler optimizations. Third, OATest incorporates two test oracles (*i.e.*, crash and inconsistency) to determine whether synthesized tests reveal DL compiler bugs.

We evaluated OATest through an extensive study on two widely-used DL compilers, *i.e.*, TVM [19] and ONNXRuntime [39]. OATest extracted 942 and 2, 116 patterns from documented tests in TVM and ONNXRuntime, respectively. Here, we used the tests generated by NNSmith (a grammar-based test generation technique for DL compilers) as the context provider for computation graph synthesis by OATest, which are called seed graphs for ease of presentation. Over a two-week fuzzing period, OATest uncovered a total of 56 previously unknown bugs, of which 42/24 were confirmed/fixed by developers. The bugs detected by OATest are diverse, involving a wide range of optimizations and root causes. We also compared OATest with two state-of-the-art DL compiler fuzzers (*i.e.*, WhiteFox and NNSmith) and a custom LLM-based technique we developed (*i.e.*, LLMTest). Within a 12-hour fuzzing budget for repeating 5 times, OATest detected 42 unique bugs (including 30 optimization bugs), significantly outperforming NNSmith, WhiteFox and LLMTest, which identified only 5 bugs (including 2 optimization bugs), 4 bugs (including 3 optimization bugs) and 7 bugs (including 3 optimization bugs), respectively. Moreover, OATest covered 60.20% more branches and 66.98% more lines on average compared to the three baselines regarding DL compiler optimization code, demonstrating the significant superiority of OATest over the state-of-the-art fuzzing techniques. Also, our evaluation demonstrates the efficiency of OATest and confirms the contribution of key components in OATest.

To sum up, this work makes the following major contributions:

- We introduce *computational graph synthesis* to generate optimization-aware tests for DL compiler optimizations.
- We develop a novel test generation technique (*i.e.*, OATest) guided by the idea, which extracts optimization-aware patterns from documented tests and integrates them with diverse contexts to synthesize optimization-aware computational graphs as tests.
- We conduct an extensive study to evaluate OATest based on two widely-used DL compilers (*i.e.*, TVM and ONNXRuntime), demonstrating the superiority of OATest over two state-of-the-art DL compiler fuzzing techniques by detecting 56 previously unknown bugs with 42/24 confirmed/fixed.

The source code of OATest and all experimental data have been released for replication and future use, which are accessible at: **https://github.com/ShenQingchao/OATest.**

2 Preliminaries

2.1 DL Compilers

DL compilers process DL models to produce optimized, deployable code for diverse hardware. In general, DL compilers involve two main stages. First, the *model loading* stage is responsible for converting DL models built from various DL libraries (*e.g.*, PyTorch [4] and TensorFlow [48]) into Intermediate Representations (IRs). DL models and IRs are two distinct formats of computational graphs: the former is designed for inference while the latter is designed for optimization. Regardless of the formats, a computation graph is a directed acyclic graph, where nodes represent operators (*e.g.*, Conv2D), edges represent data dependencies between operators, and tensors flow along edges as inputs and outputs of operators. That is, the variable on the edge is a tensor representing the input or output of an operator. Then, the *optimization* stage applies a sequence of optimizations (also called *passes*) to the IR for transformation in order to enhance its performance.

Optimizations in DL compilers can be classified into two categories: block-level optimization (*e.g.*, algebraic simplification) and subgraph-level optimization (*e.g.*, common subexpression elimination) to reduce redundancy and enhance efficiency [31]. The optimization can only be executed when the computational graph contains a specific structure that matches the predefined conditions in the optimization (*i.e.*, pattern). For example, when a computational graph contains a (*e.g.*, `Add+ReLU`) structure that matches the

```
1  @I.ir_module
2  class Module:
3     @R.function
4     def predict(x: R.Tensor((10,), dtype="float32")) -> R.Tensor((10,), dtype="float32"):
5        with R.dataflow():
6           out : R.Shape([10,]) = R.add(x, x)
7           lv = R.tensor_to_shape(out)
8           alloc4 = R.builtin.alloc_tensor(lv, runtime_device_index=0)
9           R.output(alloc4)
10       return alloc4
```

(a) The synthesized computational graph

```
1  @I.ir_module
2  class Module:
3     @R.function
4     def main(A: R.Tensor((2, 4), dtype="float32")) -> R.Tensor((2, 4), dtype="float32")
5        lv: R.Shape([2,4]) = R.tensor_to_shape(A)
6        alloc4: R.Tensor((2,4), dtype="float32") = R.builtin.alloc_tensor(
7                  lv, runtime_device_index=0
8        )
9        return alloc4
```

(b) Computational graph from the TVM documented test

```
1  @I.ir_module
2  class Module:
3     @R.function
4     def predict(x: R.Tensor((10,), "float32")): -> R.Tensor((10,), dtype="float32"):
5        with R.dataflow():
6           out : R.Shape([10,]) = R.add(x, x)
7           R.output(out)
8        return out
```

(c) Seed graph

```
1  lv = R.tensor_to_shape(A)
2  alloc4= R.builtin.alloc_tensor( lv, runtime_device_index=0)
```

(d) The abstract pattern

Figure 1: Motivation example

pattern of `OpFusion` *pass*, the *pass* will fuse the two operators into one to form a more optimized version. Finally, the optimized IR is compiled to generate deployable code for diverse hardware.

Unlike traditional compilers (*e.g.*, LLVM [28] and JVM [44]), DL compilers take as input computational graphs instead of imperative programs and execute various DL-specific optimizations (*e.g.*, operator fusion). Consequently, fuzzing techniques [29, 55, 58] for traditional compilers cannot be applied to DL compilers due to significant differences in input formats and test objectives [42, 43].

2.2 A Motivating Example

Figure 1(a) shows a computational graph generated by OATest that detects a previously unknown bug in TVM [11]. This bug in the `StaticPlanBlockMemory` *pass* stems from the incorrect handling for the `DataflowBlockNode` instance (*i.e.*, `R.builtin.alloc_tensor`). The `StaticPlanBlockMemory` *pass* is a block-level optimization that enhances memory allocation by analyzing usage patterns and lifetimes for static planning and reuse. However, when utilizing the `StaticPlanBlockMemory` *pass* to optimize the synthesized graph, TVM crashed unexpectedly. After reporting to developers, it has been fixed by introducing handling for the `DataflowBlockNode`, ensuring proper processing of blocks during memory planning [15].

This bug is difficult to trigger by existing techniques because it requires strict conditions, including (1) a specific pattern (*i.e.*, a static computational graph with the `R.builtin.alloc_tensor` operator) for triggering the optimization and (2) an extra condition (*i.e.*, the pattern must be wrapped in a dataflow block – the `R.dataflow()` construct in Line 5 of Figure 1(a)) to cover the buggy code. Among 1,000 tests generated by NNSmith, none triggered this optimization, confirming the limited effectiveness of such a grammar-based technique. Also, WhiteFox fails to extract complete triggering conditions and always produces invalid tests due to LLMs' hallucination, thereby missing this bug.

We noticed that the tests documented in TVM include the specific pattern but fail to reveal the bug because they lack the second context condition, resulting in the buggy code remaining uncovered. A bug-revealing computational graph (*i.e.*., Figure 1(a)) can be generated by synthesizing the optimization-aware pattern extracted from the documented test (*i.e.*., Figure 1(b)) with a seed graph generated by NNSmith (*i.e.*., Figure 1(c)). Note that the seed graph also cannot trigger the bug due to lacking the pattern for triggering the `StaticPlanBlockMemory` *pass*. Specifically, we first extract the optimization-aware pattern (*i.e.*, Lines 5-8 in Figure 1(b)) from the computational graph collected from the TVM documented test. Then, we synthesize a new test by combining the pattern with a seed graph that contains the extra condition. During the synthesis process, a technical challenge is to establish effective connections between the pattern and the context and thus generate a valid test. Specifically, direct insertion would result in an undefined input for `R.tensor_to_shape` (*i.e.*, the variable `A`), which has a shape constraint of `(2,4)`. Moreover, we cannot find a compatible variable in the seed graph that satisfies the shape constraint for fixing the undefined inputs. Our empirical evaluation revealed that direct test insertion achieved merely 5% validity rate during a 12-hour fuzzing process. To solve it, we abstract the extracted pattern into an abstracted one to loosen the constraints by eliminating all specific shape and type values in the inputs/outputs of nodes (*i.e.*., Figure 1(d)) and fix the undefined input (*e.g.*, parameters of `A`) in the abstract pattern by reusing the input of the existing node (*e.g.*, the `out` variable in Figure 1(c)) in the seed graph. Note that if none of the variables in the context satisfy the connection constraints, we will create bridge nodes to link the undefined input in the pattern to the seed graph , thereby forming a valid computational graph (will be introduced in Section 3.2.2). In this way, the bug-revealing computational graph is finally generated (as shown in Figure 1(a)).

Intuitively, mutation-based test generation that mutates the documented tests may also generate tests to fuzz DL compiler optimizations. However, no mutation-based technique exists specifically for DL compiler testing. Traditional mutation (*e.g.*, minor changes for the parameter settings of operators) is less effective here due to DL compilers' coarse-grained blocks or subgraphs optimizations in computational graphs. This restricts their ability to cover new optimization behaviors, reducing their effectiveness in fuzzing DL compiler optimizations [43]. In summary, embedding optimization-aware patterns into diverse contexts can be more effective in generating optimization-aware tests.

3 Approach

In this section, we present the methodology of OATest, our novel DL compiler optimization fuzzing technique, which aims to synthesize computational graphs by integrating optimization-aware

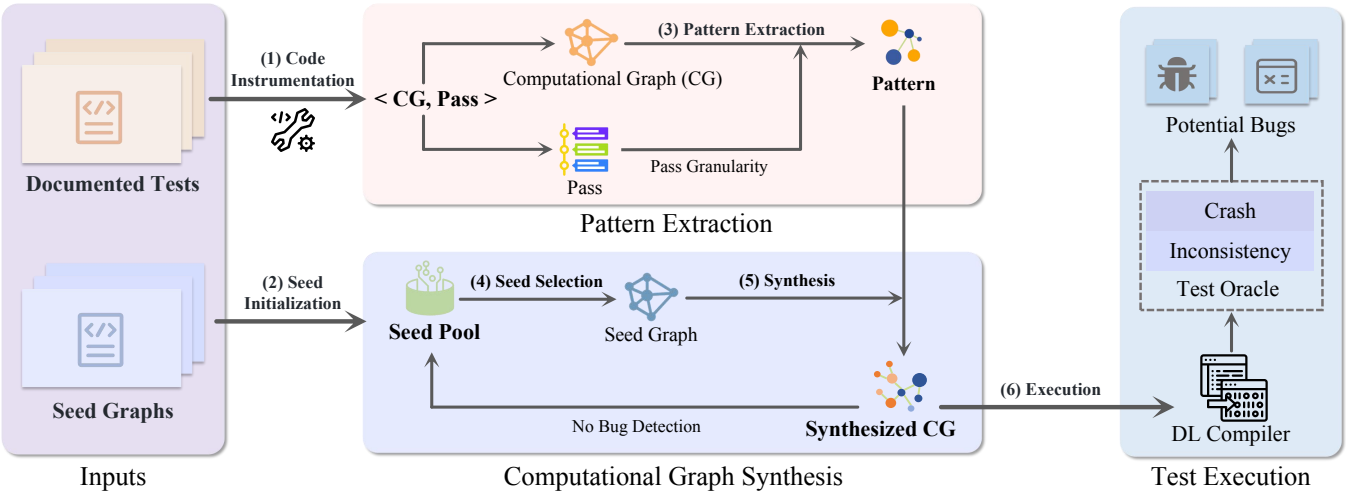

Figure 2: The Overview of OATest

patterns obtained from documented tests into diverse contexts. As illustrated in Figure 2, OATest comprises three primary steps. First, OATest extracts optimization-aware patterns from computational graphs by instrumenting code in documented tests. As optimizations in DL compilers work at different granularities (*i.e.*, block and subgraph), patterns are identified by extracting blocks or subgraphs accordingly as needed to ensure optimization awareness (Section 3.1). Second, OATest synthesizes a computational graph by incorporating an extracted pattern into a given seed graph, where the seed graph serves as context (Section 3.2). In particular, OATest fixes the undefined inputs of operators in the pattern by either (1) utilizing the compatible inputs/outputs of existing nodes in the context or (2) creating nodes with compatible inputs/outputs as a bridge for connection. Third, OATest incorporates two test oracles to run the test and determine whether a synthesized computational graph uncovers a DL compiler bug. Bug-free computational graphs generated by OATest will be added to the seed pool as a new seed graph for further synthesizing with pattern combination (Section 3.3).

3.1 Pattern Extraction

In documented tests of DL compilers, diverse optimization-aware computational graphs (CGs) were crafted to test each optimization's implementation. Since different optimizations work at different granularities, extracting optimization-aware patterns from the documented tests involves two steps: (1) identifying the computational graph and its corresponding optimization (denoted as $\langle CG, pass \rangle$ pair), and (2) extracting optimization-aware patterns from the computational graph based on the granularity of the optimization.

First, OATest collects the $\langle CG, pass \rangle$ pairs from documented tests. In DL compilers, documented tests only include computational graph generation statements, instead of a concrete computational graph. These statements dynamically generate a computational graph in the memory during test execution. Since static analysis is insufficient for capturing such runtime-generated computational graphs, OATest utilizes code instrumentation to dynamically gather all $\langle CG, pass \rangle$ pairs during test execution in DL compilers. Specifically, to capture all $\langle CG, pass \rangle$ pairs in the documented tests, OATest hooks into the *pass* call function in the base class of *pass*, which is inherited by all specific *passes*. This function allows access to both the *pass* object and the computational graph being optimized. Once the hooked API is called, the computational graph to be optimized and the invoked optimization will be collected simultaneously as a $\langle CG, pass \rangle$ pair, respectively.

For each collected $\langle CG, pass \rangle$ pair, OATest further extracts the optimization-aware patterns from the computational graph based on the granularity of the associated optimization. Since each optimization is thoroughly described in the documentation of the corresponding DL compiler, we identify its granularity by analyzing the corresponding description and categorizing it as either block-level or subgraph-level. Specifically, OATest begins by constructing a dependency graph that captures the relationships between nodes (*i.e.*, operators) in the computational graph. It then traverses the graph to identify the corresponding patterns. For subgraph-level optimization, OATest identifies distinct computational units as subgraph patterns by grouping operators that share input or output tensors. For block-level optimization, OATest further groups operations within the subgraph into block patterns based on shared intermediate tensors or execution order dependencies. A subgraph is a connected, semantically complete computational unit within a computational graph, with explicit input/output boundaries and executable isolation, while a block is a structured, contiguous region of a subgraph defining a localized computation. This method efficiently decomposes the computational graph into meaningful optimization-aware patterns prepared for subsequent computational graph synthesis.

3.2 Computational Graph Synthesis

During the computational graph synthesis process, OATest randomly selects a pattern and a seed graph, then inserts the pattern into the seed graph by randomly choosing the synthesis point. These random selections contribute to the diversity of the synthesized computational graphs. However, simply putting them together will lead to invalid tests due to the undefined inputs for the operators in patterns. To solve this, OATest designs two synthesis

strategies, *i.e.*, *reusing the compatible inputs/outputs of existing nodes in contexts* and *creating new nodes with compatible inputs/outputs*. These strategies synthesize valid, optimization-aware computation graphs by leveraging connections between patterns and contexts. The technical details are introduced in the following.

3.2.1 Reusing Existing Node. For each undefined input from the nodes in patterns, OATest prioritizes reusing compatible inputs/outputs of the existing nodes in the context to establish new connection edges, ensuring strong interactions between patterns and contexts. Edge constraints in a computational graph are highly strict, requiring that the output of a node (*i.e.*, a tensor variable) must align in type and shape with the input of the subsequent node. In fact, it is less likely to find a compatible input/output of a node from the context that satisfies the strict constraints for the edge connecting an undefined input of the node in the pattern. If no compatible input/output of nodes is found, OATest loosens the constraints by removing specific shape and type values and retaining only the intrinsic constraints of each node (*e.g.*, the input of `Conv2D` must be a 4-dimensional tensor). This abstraction increases synthesis flexibility without reducing optimization awareness, as DL compiler optimizations mainly depend on graph structure rather than the specific tensor values. Figure 1(d) shows an abstract pattern derived from the pattern in Lines 5-8 of Figure 1(b). The abstract pattern eliminates the constraints for the input of the node `R.tensor_to_shape` (*i.e.*, `R.Shape([2,4])`) in the concrete pattern, allowing a tensor with other shapes to be a valid input.

To merge the abstract pattern into the seed graph, OATest first randomly selects a synthesis point in the seed graph, which can improve the diversity of the generated tests. All outputs of nodes before the synthesis point form a preceding context, acting as a variable pool for repairing undefined inputs. Specifically, for each undefined input in the abstract pattern, OATest searches the preceding context for compatible variables satisfying operator constraints (*e.g.*, the inputs/outputs shape and type constraints). The operator constraints are reused from the source code of NNSmith [34]. If a compatible variable is found, OATest establishes an edge connection between the seed graph and the abstract pattern by assigning the selected variable as the input of the node with an undefined input. Meanwhile, to ensure that the context fully encompasses the abstract pattern, OATest further creates connections between the broken output edge in the pattern and the context. Similarly, for each broken output edge, OATest searches from the succeeding context, a variable pool with all outputs of nodes after the synthesis point, for a compatible input of node in the seed graph and creates a connection by assigning the variable in the broken output edge as the input of the selected node with compatible input. Finally, a valid optimization-aware computational graph is generated.

For example, in Figure 3, OATest first randomly selects the synthesis point (①) and classifies inputs/outputs of nodes in the seed graph into the preceding context or the succeeding context. Then, for the undefined input of nodes in the pattern (*i.e.*, the input of the `Conv2D` operator in the abstract pattern), OATest selects a compatible variable (*e.g.*, the `bias_add_output`) from the preceding context and creates a new connection between them (②). Similarly, for the broken output edge (*i.e.*, `add_output`), OATest links them to compatible inputs (of the `Softmax`) in the succeeding context

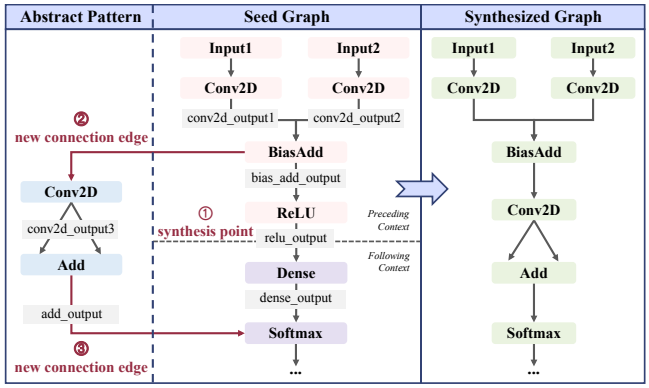

Figure 3: Fixing undefined inputs by reusing inputs/outputs of existing nodes in the context

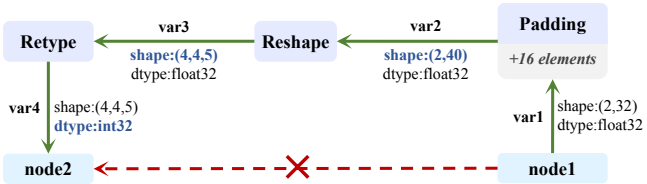

Figure 4: Fixing undefined inputs by creating new nodes

(③). Finally, the abstract pattern is well merged into the seed graph, generating a new synthesized computational graph.

3.2.2 Creating New Nodes. Not all undefined inputs in the pattern can find a compatible tensor variable in the seed graph, even in the abstract pattern with loosened constraints. If no compatible tensor variable is found in the context, the aforementioned strategy of reusing existing nodes does not work. In this situation, OATest creates new nodes in the seed graph as bridges between the pattern and the seed graph. Specifically, OATest randomly selects a tensor variable from the preceding context and compares its attributes (*i.e.*, shape and type) with the constraints of the undefined input. If the shapes are inconsistent, OATest first adjusts the element count (*i.e.*, total number of the data point) of the selected tensor variable by inserting a `Padding` or `Cropping` node, generating a new tensor variable that satisfies the element count constraints for the undefined input. Subsequently, when the new tensor variable has different tensor dimension with the undefined input, OATest appends a `Reshape` node to align the tensor dimensions (*i.e.*, the number of axes) with those of the undefined input, thereby producing a shape-compatible tensor variable. Additionally, when the types differ, OATest applies a `Retype` node to perform explicit type conversion to the type of the undefined input, ensuring the adjusted tensor variable satisfies the type inconsistency of the pattern. Finally, the compatible tensor variable is assigned to the undefined input, forming a new connection edge and thus synthesizing a new computational graph. The same method is also used for fixing the broken output edge in the pattern.

Figure 4 shows an example of creating new nodes for fixing the undefined input `node2`. OATest randomly selects a variable (*i.e.*, `var1`) from the context. The selected tensor variable `var1` has a shape of `(2,32)` and a type of `float32`. However, the undefined

input node2 expects a tensor variable with shape (4,4,5) and type int32 as input. To align the shapes, OATest applies a Padding operator to create equal-sized var2 followed by a Reshape operator to produce shape-compatible var3. To further align the type constraint, OATest appends a Retype operator to convert the type of var3 from float32 into int32 and obtain the type-compatible variable var4, which is also shape-compatible. Finally, OATest establishes a new connection between the node2 in pattern and the seed graph by assigning the var4 as the input of node2.

3.3 Test Oracle

A recent study found that most DL compiler bugs manifested as crashes and inconsistent inference results [42]. Hence, we employ two test oracles in OATest to test DL compilers.

Crash refers to the unexpected termination of the compilation process. It is a widely-used test oracle in DL compiler testing [34, 35, 37]. According to a recent bug study in DL compilers [42], bugs with crash symptoms occupy 59.37% of all studied bugs, showing the significant importance of detecting crash bugs. Note that crashes that produce error messages like "unsupported" are disregarded, as they usually indicate unsupported features rather than actual bugs.

Inconsistency refers to situations where a DL compiler yields inconsistent inference results for the same inputs with different optimizations or hardware devices (*e.g.*, CPU and CUDA). Since floating-point operations in DL models always cause slight differences in results rather than a bug, we measure inference discrepancies using the widely-used Chebyshev distance [20] with a threshold of 1e-3 to evaluate transformation equivalence in multi-layer networks, accounting for expected numerical error accumulation following existing work [30, 34, 37]. This threshold is also consistent with industry practices, such as TVM's official test suite for model optimization correctness validation. Since DL compilers are designed to maintain functional equivalence, any inconsistencies (exceeding the threshold) in inference results indicate a potential compiler bug.

If the synthesized computational graph passes testing without errors, OATest adds it to the seed pool for high-order synthesis. This enables combining patterns from different optimization-related tests, facilitating the creation of more complex optimization scenarios to uncover deeper optimization bugs.

4 Evaluation

4.1 Setup

Our study aims to address the following research questions (RQs).

- **RQ1**: How does OATest perform in detecting previously unknown optimization bugs in DL compilers?
- **RQ2**: How does OATest compare against the state-of-the-art DL compiler fuzzers?
- **RQ3**: How does each key component in OATest contribute to the overall effectiveness?
- **RQ4**: How efficient does OATest in generating valid tests?

4.1.1 Subjects. Following recent DL compiler fuzzing works [34, 37, 53], we chose the widely-studied DL compilers, *i.e.*, **TVM** [19] and **ONNXRuntime** [39], as the subjects in our study. TVM is an end-to-end compiler that can optimize and deploy DL models built on various DL frameworks across diverse platforms. It contains 24 block-level optimizations and 41 subgraph-level optimizations implemented across over 140 source files. We chose the latest version of TVM (commit id: 292ecfd) as the subject under test to carry out the evaluation of OATest. ONNXRuntime is a graph-optimized DL compiler designed for ONNX models, the most widely-adopted model format. Nearly all pre-trained DL models, developed using various frameworks (*e.g.*, PyTorch [4] and TensorFlow [48]), can be converted to ONNX format for executing diverse optimizations with ONNXRuntime. It includes over 21 block-level optimizations and 25 subgraph-level optimizations implemented across over 130 source files. We chose the latest version of ONNXRunutime (commit id: 5c1b7cc) as the subject under test. These DL compilers involve two representative computational graph formats, *i.e.*, Relax IR and ONNX IR, which are compatible with nearly all mainstream DL compilers, including TVM [19], MLC-LLM [47], ONNXRuntime [39], TensorRT [2], OpenVINO [1], and Glow [41]. Hence, OATest implementation can be directly used for fuzzing other compatible DL compilers, indicating the high generalizability of our tool.

4.1.2 Baselines. We compared OATest with two state-of-the-art DL compiler fuzzers (*i.e.*, **NNSmith** [34] and **WhiteFox** [54]) as well as a custom LLM-based technique we developed (denoted as **LLMTest**). NNSmith is a grammar-based computational graph generator that produces diverse and valid computational graphs for testing DL compilers. It ensures the validity of computational graphs through constraint solving. WhiteFox is a white-box fuzzer built upon LLMs. It utilizes an analysis LLM to extract optimization triggering conditions from source code and a generation LLM to create computational graphs aligned with these specific conditions. LLMTest puts documented tests and corresponding optimization descriptions into the prompt and then instructs LLMs to generate tests to sufficiently test the optimization. Building upon previous work [54], the method applies few-shot learning with identical configuration parameters to WhiteFox. To evaluate the effectiveness of different LLMs in generating optimization-aware tests, we implemented this technique using two different state-of-the-art LLMs (*i.e.*, DeepSeek-v3 [33] and Qwen2.5-Coder [27]), which are referred to as LLMTest_{DS} and LLMTest_{Qwen}, respectively. Other DL compiler testing techniques, either cannot test the compiler optimizations for computational graphs (*i.e.*, OPERA [43] and TZer [35]) or have been demonstrated less effective than NNSmith or WhiteFox (*i.e.*, MT-DLComp [53], GenCoG [51], and HirGen [37]). Therefore, we do not include them as baselines for comparison.

4.1.3 Metrics. We mainly targeted the following metrics for evaluation. First, we used **the number of detected bugs** to measure the effectiveness of a DL compiler fuzzing technique following the existing work [34, 36, 37, 43, 46]. During the fuzzing process, a lot of test failures might be produced, but many of them might be caused by the same bugs. Therefore, we performed de-duplication on them. For crash bugs, we de-duplicated them based on the crash messages following the existing work [43, 45, 50]. For each inconsistency bug, we identified the minimal set of optimizations that could still reveal the bug based on the idea of delta debugging [49, 56]. If two inconsistency bugs had the same minimal sets of optimizations, we regarded them as duplicates. Each distinct bug was submitted to developers for further diagnosis. Here, we measured the number of submitted bugs after de-duplication, and the number of confirmed/fixed bugs.

Also, we used **code coverage** as another metric to assess OATest's effectiveness following the existing work [34, 35, 50, 52]. Since OATest was designed for fuzzing DL compiler optimizations, we measured the line and branch coverage with regard to the source code of DL compiler optimizations. To evaluate OATest's efficiency in generating valid tests in RQ4, we measured **the number of valid tests** generated by it within a given fuzzing time budget.

4.1.4 Implementations. OATest collected optimization-aware patterns from the documented tests of TVM and ONNXRuntime (*i.e.*, all tests in the folder of `test/tvm/relax/transform` for TVM and all tests in the folder of `onnxruntime/test/optimizer` for ONNXRuntime). Both TVM (65 optimizations) and ONNXRuntime (46 optimizations) achieve 100% optimization coverage, with each optimization supported by multiple documented tests. In total, we obtained 942 and 2,116 patterns for TVM and ONNXRuntime, respectively. Since DL compilers (including TVM [19], ONNXRuntime [39], OpenVINO [1], TorchInductor [18], Glow [41], and nGraph [21]) always provide well-documented tests for optimizations, the availability of such tests does not limit OATest.

We then collected a set of tests generated by NNSmith as seed graphs for OATest. Note that OATest is not specific to this source of seed graphs. We selected NNSmith rather than WhiteFox as the seed provider since the latter required much more costs for generating tests due to heavily depending on LLMs. For NNSmith, it is efficient to generate a large number of tests, and thus we collected 4k tests (taking only 10 minutes for generation) as seed graphs in our study, which could provide diverse contexts for test synthesis. All these seed graphs did not trigger bugs.

We directly adopted the implementation of NNSmith released by the authors [34] for comparison. For WhiteFox, we adapted the code for compatibility with TVM and ONNXRuntime. Furthermore, we adopted Gcov [23] and Lcov [40] to collect and analyze the line and branch coverage achieved by each technique.

4.1.5 Process. In RQ1, we ran OATest for two weeks on each DL compiler to sufficiently evaluate its effectiveness in detecting previously unknown bugs. In the remaining RQs, we ran each studied technique for 12 hours (including both test generation and execution time) on each compiler following the existing work [43, 50] for fair comparison. Particularly, we repeated each experiment for five times and aggregated the bug detection results for comparison, in order to reduce the influence of randomness involved in the fuzzing process. Note that OATest takes 4k tests generated by NNSmith (within 10 minutes) as seeds, while NNSmith ran for 12 hours, making comparison between them meaningful. All experiments were conducted on a server with an Intel Xeon Gold CPU, four A800 GPUs, and 500 GB of RAM, operating on a 64-bit Ubuntu 16.04 OS.

4.2 RQ1: Previously Unknown Bugs Detected by OATest

During two-week fuzzing, OATest detected 56 previously unknown bugs, of which 42/24 have been confirmed/fixed by developers. The remaining bugs are still awaiting developers' feedback. Among the 56 bugs, 40 are TVM bugs and 16 are ONNXRuntime bugs. Table 1 shows the details of all confirmed bugs detected by OATest, where Columns 1-6 represent the buggy compiler, the issue ID, the bug symptom, the root cause of the bug, the buggy optimization, and the bug status labeled by developers, respectively. Particularly, some of our reported bugs are regarded as "high-quality" by the TVM community and our contributions have been appreciated [6, 9, 10]. These confirmed bugs cover a wide range of buggy optimizations, symptoms, and root causes. In the following, we analyzed them from the three aspects.

Table 1: Previously unknown Bugs found by OATest

Compiler	Issue	Symptom	RC	Optimization	Status
TVM	#17120	Crash	ICL	MergeCompositeFunctions	Fixed
TVM	#17121	Crash	TSP	DeadCodeElimination	Fixed
TVM	#17175	Crash	IEH	LegalizeOps	Fixed
TVM	#17176	Crash	ICL	AttachGlobalSymbols	Fixed
TVM	#17200	Crash	ICL	LiftTransformParams*2	Fixed
TVM	#17205	IC	ICL	RealizeVDevice*2	Fixed
TVM	#17210	Crash	ICL	MergeCompositeFunctions	Fixed
TVM	#17211	Crash	TP	-	Fixed
TVM	#17213	Crash	ICL	RealizeVDevice*2	Fixed
TVM	#17215	Crash	TSP	FlattenBuffer	Fixed
TVM	#17217	Crash	TSP	VMBuiltinLower	Fixed
TVM	#17218	Crash	TSP	VMBuiltinLower	Fixed
TVM	#17222	Crash	ICL	DeadCodeElimination	Fixed
TVM	#17223	Crash	IEH	-	Confirmed
TVM	#17231	Crash	ICL	LiftTransformParams	Fixed
TVM	#17235	Crash	TP	-	Fixed
TVM	#17242	Crash	IEH	-	Fixed
TVM	#17243	Crash	TP	-	Fixed
TVM	#17249	IC	ICL	-	Fixed
TVM	#17253	IC	ICL	RemoveUnusedOutputs	Fixed
TVM	#17254	Crash	TP	ToMixedPrecision	Fixed
TVM	#17269	Crash	ICL	LazyTransformParams	Confirmed
TVM	#17311	Crash	IEH	-	Confirmed
TVM	#17340	Crash	ICL	KillAfterLastUse, FoldConstant	Confirmed
TVM	#17341	Crash	ICL	AnnotateTIROpPattern, FuseOps, FuseTIR	Confirmed
TVM	#17348	Crash	ICL	StaticPlanBlockMemory	Confirmed
TVM	#17357	Crash	ICL	FuseTIR, LambdaLift, AllocateWorkspace	Confirmed
TVM	#17370	Crash	IEH	LegalizeOps	Fixed
TVM	#17483	Crash	ICL	LegalizeOps	Fixed
TVM	#17486	Crash	ICL	LegalizeOps	Fixed
TVM	#17487	Crash	ICL	-	Confirmed
TVM	#17488	Crash	ICL	StaticPlanBlockMemory	Fixed
ORT	#23086	Crash	IEH	FuseQuickGeLU	Confirmed
ORT	#23114	Crash	ICL	FusedConv	Confirmed
ORT	#23116	Crash	TP	FuseReluClip	Confirmed
ORT	#23118	Crash	ICL	DeadCodeElimination	Confirmed
ORT	#23119	Crash	IEH	FusedConv	Confirmed
ORT	#23138	Crash	ICL	GeluFusion	Confirmed
ORT	#23143	IC	ICL	MemCpy	Confirmed
ORT	#23199	IC	ICL	Quantizer, Dequantizer	Confirmed
ORT	#23258	Crash	ICL	Dequantizer	Fixed
ORT	#23284	IC	ICL	-	Confirmed

RC: Root Cause; **ORT**: ONNXRuntime; **IC**: Inconsistency; **ICL**: Incorrect Code Logic; **TSP**: Tensor Shape Problem; **IEH**: Incorrect Exception Handling; **TP**: Type Problem;

Buggy Optimization Analysis: As OATest is designed for fuzzing DL compiler optimizations, we first investigated whether each confirmed bug is an optimization bug and labeled the corresponding buggy optimizations based on the feedback from developers. From this table, 33 **out of** 42 **confirmed bugs detected by OATest are optimization bugs**, including 24 TVM bugs and 9 ONNXRuntime bugs. The 33 optimization bugs involve 26 optimizations, including 28 bugs involving a single optimization and 5 bugs involving multiple optimizations. The remaining 9 bugs stem from flawed IR design in the optimization stage, not faulty optimizations. Since IRs serve as the object for optimization, identifying such flaws

```
  void UpdateGroupDependencies(Group* group, const Array<Expr>& args) {
    Group* group_root = group->FindRoot();
-   for (const auto& arg : args) {
-     auto arg_group_root = memo_[arg]->FindRoot();
+   std::function<void(Expr)> visit_expr = [&](Expr expr) {
+     if (expr.as<GlobalVarNode>()) return;
+     if (auto tuple = expr.as<TupleNode>()) {
+       for (const auto& field : tuple->fields) {visit_expr(field);}
+       return;
+     }
+     ...
+     auto arg_group_root = memo_[expr]->FindRoot();
```

Figure 5: A crash bug detected by OATest (#17120). The bug is caused by Incorrect Code Logic

is also crucial. The results demonstrate the effectiveness of OATest in uncovering previously unknown optimization bugs.

For example, a test created by OATest exposes an inconsistency bug [5] specifically when the `RealizeVDevice` *pass* is executed twice. Due to extensive code changes, this bug is not shown as a figure in this paper and more details can be found in the issue [5]. Specifically, the `RealizeVDevice` *pass* illegally mutated the relax expression in place. Consequently, applying `RealizeVDevice` a second time caused unintended updates in unrelated expressions, resulting in incorrect IRs. To address this, a complex patch[5] was introduced, refactoring the `RealizeVDevice` *pass* to eliminate all in-place mutations with more than 600 lines of code changes.

Symptom Analysis: The symptoms of our detected bugs include crashes and inconsistency. *Crashes* indicate unexpected termination during the compilation process of DL compilers, with 36 out of the 42 bugs exhibiting this symptom. *Inconsistency*, observed in 6 confirmed bugs, refers to producing inconsistent inference results or IRs during the differential testing process. These bugs pose significant risks as they can *silently* produce incorrect optimization results. The results demonstrate the effectiveness of both test oracles in OATest.

Figure 5 shows a crash bug in TVM [8]. A computational graph synthesized by OATest triggered this bug since it contains the optimization-aware pattern (*i.e.*., fused operators) for activating the `MergeCompositeFunctions` *pass* and another bug-triggering condition (*i.e.*, `R.call_tir` operator) provided by context. The lack of checks for the `relax::GlobalVar` referenced by `R.call_tir` in the `MergeCompositeFunctions` *pass* leads to null pointer access, resulting in a crash. This bug was fixed by adding type checks to avoid null pointer dereference [13]. However, detecting this bug is challenging for NNSmith and WhiteFox. NNSmith rarely triggers the `MergeCompositeFunctions` *pass* through random grammar-based fuzzing, while WhiteFox struggles to generate a valid computational graph for triggering this *pass*.

Root Cause Analysis: According to the discussion from developers and the patch for the fixed bugs, we identified four distinct root causes behind the 42 confirmed bugs detected by OATest, classified following a recent study on DL compiler bugs [42]. The results demonstrate that OATest is capable of revealing bugs with diverse root causes, including Incorrect Code Logic, Tensor Shape Problem, Incorrect Exception Handling, and Type Problem.

Incorrect Code Logic refers to the bugs arising from the incorrect algorithm logic (*e.g.*, operator fusion), requiring modification with

```
  class LowerRuntimeBuiltinMutator : public ExprMutator {
    ...
      return Reshape(call);
    } else if (call->op == shape_of_op_) {
      return ShapeOf(call);
+   } else if (call->op == tensor_to_shape_op_) {
+     return TensorToShape(call);
```

Figure 6: A crash bug detected by OATest (#17217). The bug is caused by Tensor Shape Problem

many statements. 26 bugs (61.90% of the confirmed/fixed bugs) fall into this category. Depending on the bug location, this category is further divided into *Incorrect Optimization Code Logic* and *Incorrect Non-Optimization Code Logic*. The former represents a specialized type of optimization bugs caused by incorrect optimization code logic, with 20 bugs belonging to this subcategory, accounting for 60.61% of optimization bugs. The results demonstrate that most of the bugs detected by OATest have broad harm to the DL compiler, especially for the optimization code.

Tensor Shape Problem involves the bugs caused by wrong tensor shape operation, such as tensor shape inference and tensor shape transformation. Tensor shape plays a vital role in DL compiler optimizations. 4 confirmed bugs belong to this category. Figure 6 shows a Tensor Shape Problem bug [7]. The `ShapeStructInfo` type is an acceptable input type for the `shape` argument of the `Reshape` operator. However, the `Reshape` operator mistakenly required the `shape` argument to be either an inline `relax::ShapeExpr` or a variable bound to one. A test generated by OATest including a `Reshape` operator with the `shape` argument generated by `tensor_to_shape_op`, triggered this bug and crashed unexpectedly. After reporting to developers, it was fixed with a patch [12] that applied the `TensorToShape` operation to support reshaping for the `ShapeStructInfo` type.

Incorrect Exception Handling refers to the wrong management on exception. 7 confirmed bugs belong to this category. Bugs in this category pose risks for compilers as they facilitate side channel attacks [22], enabling attackers to deduce the internal state of a computation without direct access.

Type Problem refers to bugs involving type-related problems, such as type conversion and type inference. This category includes 5 confirmed bugs: 3 stemming from the tensor type problem, and 2 caused by the conventional data type problem.

4.3 RQ2: Comparison with State-of-the-art Techniques

We then compared OATest with two state-of-the-art DL compiler fuzzing techniques (*i.e.*, NNSmith and WhiteFox) and a custom LLM-based technique we developed (*i.e.*, LLMTest). Under the same setup (presented in Section 4.1.5), OATest detected 29 (including 22 optimization bugs) and 13 (including 8 optimization bugs) previously unknown bugs in TVM and ONNXRuntime respectively, whereas NNSmith detected 5 (including 2 optimization bugs) and 0 bugs , WhiteFox detected 3 (including 3 optimization bugs) and 1 (including 0 optimization bugs) bugs, LLMTest$_{DS}$ detected 4 (including 2 optimization bugs) and 2 (including 1 optimization bugs) bugs, and LLMTest$_{Qwen}$ detected 5 (including 2 optimization bugs) and 2 (including 1 optimization bugs) bugs, respectively. Among

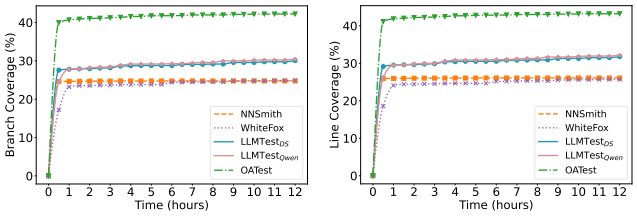

Figure 7: Branch and line coverage of TVM

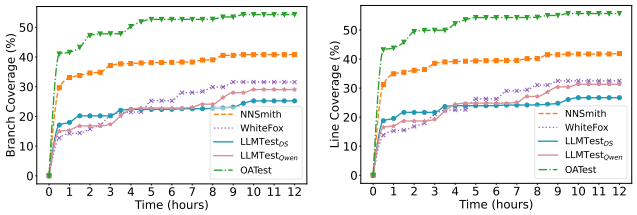

Figure 8: Branch and line coverage of ONNXRuntime

those bugs detected by baselines, only three are not detected by OATest, all of which occur in the model loading stage rather than compiler optimizations. The results demonstrate the superiority of OATest over four baselines in DL compiler fuzzing, particularly in detecting optimization bugs.

To further understand the superiority of OATest in bug detection, we compared the line coverage and branch coverage achieved by each studied technique. Figures 7 and 8 present the coverage growth within 12 hours on TVM and ONNXRuntime, respectively. From these figures, OATest consistently achieves higher branch and line coverage than four baselines on both TVM and ONNXRuntime. For example, compared to the more effective baseline in terms of code coverage (*i.e.*, LLMTest$_{Qwen}$ in TVM and NNSmith in ONNXRuntime), OATest covers 32.21% and 32.29% more branches as well as 32.29% and 32.89% more lines in the optimization code for TVM and ONNXRuntime, respectively. That is, it outperforms the more effective baseline by 60.20% in branch coverage and 66.98% in line coverage on average. The results demonstrate the capability of OATest in improving code coverage, thereby contributing to its significant bug detection effectiveness.

While WhiteFox, LLMTest$_{DS}$, and LLMTest$_{Qwen}$ are designed to test optimization in DL compilers, they exhibit limited bug detection capabilities and poor code coverage. Different LLM implementations of LLMTest achieve similar effectiveness, and employing alternative LLMs does not significantly improve performance. Our analysis traces these shortcomings to two key factors: low validity rates and inefficiency, which we further examine in Section 4.5.

4.4 RQ3: Contribution of Each Key Component

Figure 9: Venn graph of bugs detected by different techniques

Table 2: Bugs detected by OATest and variants

	OATest$_{block}$	OATest$_{subgraph}$	OATest$_{graph}$	OATest$_{noopt}$	OATest
TVM	14	20	11	7	29
ONNXRuntime	8	7	5	3	13
Total	22	27	16	10	42

4.4.1 Contribution of Different Seed Graphs. To assess the influence of seed graphs, we designed two variants of OATest: OATest$_{dt}$, which uses **d**ocumented **t**ests as seed graphs, and OATest$_{whitefox}$, which relies on tests generated by WhiteFox as seeds. Both variants used the same number of seed graphs (*i.e.*, 4k) as OATest (using NNSmith-generated tests as seeds). Figure 9 shows a Venn diagram comparing the bugs detected by the three methods with different seeds under the same experimental configuration as RQ2. OATest detected 29 bugs in TVM and 13 in ONNXRuntime. In comparison, OATest$_{dt}$ found 26 and 10 bugs, while OATest$_{whitefox}$ identified 24 and 8 bugs, respectively. Note that each technique detected several unique bugs missed by the others. Also, all of them outperform three baselines, *i.e.*, NNSmith (detected 5 and 0 bugs), WhiteFox (detected 3 and 1 bugs), and two implementations of LLMTest (detected 4 and 2 bugs in DeepSeek implementation and detected 5 and 2 bugs in Qwen implementation). These results demonstrate the effectiveness of our approach in bug detection, regardless of the seeds used, and suggest that diversifying seed graphs could further improve bug-finding capabilities.

4.4.2 Contribution of Pattern Extraction. To evaluate our pattern extraction strategy (*i.e.*, adaptively extracting block-level or subgraph-level patterns from a ⟨*CG*, *pass*⟩ pair based on the characteristics of *pass*), we first tracked how often optimization passes linked to injected patterns were triggered during test execution. Specifically, for each test generated by OATest, we executed it and collected the coverage on the source code of the target optimization. During the 12-hour fuzzing period (see Section 4.1.5), 91.36% and 75.49% of the tests synthesized by OATest successfully triggered the target optimization in TVM and ONNXRuntime, respectively. A small portion of tests generated by OATest fail to trigger the target optimization due to the random pattern injection. In comparison, WhiteFox, another optimization-aware test generation technique, achieved substantially lower optimization-triggering rates (*i.e.*, 27.79% in TVM and 21.52% in ONNXRuntime) than OATest, demonstrating OATest's effectiveness in pattern extraction.

Additionally, we further investigated the contribution of our pattern extraction strategy by constructing four variants of OATest: (1) **OATest$_{block}$** always extracts block-level patterns from each ⟨*CG*, *pass*⟩ pair, (2) **OATest$_{subgraph}$** always extracts subgraph-level patterns from each ⟨*CG*, *pass*⟩ pair, and (3) **OATest$_{graph}$** does not extract fine-grained patterns from a ⟨*CG*, *pass*⟩ pair but directly use the entire graph (*i.e.*, *CG*) for test synthesis. (4) **OATest$_{noopt}$** randomly extracts the same number of patterns as OATest from the optimization-unrelated tests documented in DL compilers. Each variant adopts a fixed pattern extraction strategy instead of the granularity-aware strategy used in OATest.

Table 2 presents the number of bugs detected by OATest and each variant during the same fuzzing time budget (introduced in Section 4.1.5). By comparing OATest with OATest$_{block}$ and OATest$_{subgraph}$, our technique detected more bugs (42) than the

```
1  class TestReturnTuple(BaseCompare):
2    @I.ir_module
3    class Before:
4      @R.function
5      def main(A: R.Tensor([16, 16], "int32")):
6        B = R.add(A, A)
7        out_tuple = Before.func(B)
8        return out_tuple
9
10     @R.function(private=True)
11     def func(B: R.Tensor([16, 16], "int32")) ->
12     R.Tuple(R.Tensor([16, 16], "int32"), R.Tensor([16, 16], "int32")):
13       C = R.multiply(B, B)
14       D = R.add(B, B)
15       return (C, D)
```

Figure 10: A bug-revealing computational graph synthesized with subgraph-level pattern

two variants (22 and 27), demonstrating the contribution of our adaptive pattern extraction strategy. By comparing OATest$_{graph}$ with OATest$_{block}$, OATest$_{subgraph}$ and OATest, the former detected the smallest number of bugs in both TVM and ONNXRuntime. Specifically, OATest$_{graph}$ detected only 11 TVM bugs and 5 ONNXRuntime bugs. The results demonstrate the contribution of extracting fine-grained patterns for test synthesis, compared to using the entire computational graph from a documented test. By comparing OATest$_{noopt}$ with OATest, the latter detects much more bugs (42) than the former (10) in total. Furthermore, OATest covered 51.24% more branches and 47.33% more lines in the optimization code than OATest$_{noopt}$ on average. These results demonstrate that the extracted patterns by OATest are more optimization-aware, allowing it to detect more optimization bugs.

Figure 10 shows a reduced bug-revealing test synthesized by OATest at the subgraph level. The `main` function is a seed graph and the `func` function is a subgraph-level pattern. Applying this *pass* to the synthesized computational graph produced an incorrect IR silently. The `RemoveUnusedOutputs` *pass* mistakenly identified a tuple element as only in `TupleGetItem` nodes, failing to account for scenarios where tuples are returned from a function. The bug can only be triggered when a function calls a subroutine that creates a tuple and then immediately returns that tuple, making it detectable only through subgraph-level synthesis. Block-level synthesis tests fail to trigger the bug because they lack the necessary subgraph structure (*i.e.*, the `func` function). Furthermore, even when the entire seed graph is used as a synthesis pattern, the bug remains undetected because the tuple produced by the subgraph is not propagated as the output of the overall graph. To fix it, the developers committed a patch [14] by modifying `RemoveUnusedOutputs` *pass* to check for tuple object usage, not just for the `TupleGetItem`.

4.4.3 Contribution of Test Synthesis. We then investigated the contribution of test synthesis (*i.e.*, synthesizing patterns into seed graphs). First, all the documented tests for optimization (used for pattern extraction) and seed graphs (served as context) used by OATest cannot detect any bugs in both TVM and ONNXRuntime, but synthesizing them with OATest detected a number of bugs presented before, demonstrating the contribution of test synthesis in bug detection. Then, we collected the line coverage and branch coverage achieved by all the documented tests and seed graphs, and analyzed whether OATest can achieve more optimization code than them. Within the given fuzzing time budget, OATest further covered 3.75% new branches and 3.57% new lines in the optimization code of TVM, and covered 5.95% new branches and 5.22% new lines in the optimization code of ONNXRuntime, compared to all the used documented tests and seed graphs, respectively. The results further confirm the contribution of test synthesis with OATest.

4.5 RQ4: Efficiency

Within the given fuzzing time budget, OATest generated and executed 41.9k and 112k valid tests for TVM and ONNXRuntime, respectively, while NNSmith generated and executed 37.2k and 103.7k valid tests for TVM and ONNXRuntime, WhiteFox generated and executed 1.6k and 2k valid tests, LLMTest$_{DS}$ generated and executed 1.7k and 1.9k valid tests, and LLMTest$_{Qwen}$ generated and executed 2.6k and 3.1k valid tests. The results demonstrate the efficiency superiority of test synthesis with OATest over grammar-based test generation with NNSmith and LLM-based test generation with WhiteFox and LLMTest. Through further analysis, the test generation process consumed less than 10% of the entire fuzzing time for OATest, but over 95% for WhiteFox and LLMTest. This indicates that the efficiency of test generation is a major bottleneck for WhiteFox and LLMTest due to LLMs' hallucinations, causing low test validity (37% for TVM, 44.36% for ONNXRuntime in WhiteFox and 42.71% for TVM, 44.54% for ONNXRuntime average in two different implementations of LLMTest) and limiting its efficiency.

5 Discussion

5.1 Threats to Validity

The threat to internal validity mainly lies in the implementation of OATest and the adaptation of WhiteFox for TVM and ONNXRuntime. To mitigate this, two authors thoroughly reviewed and tested the code. For WhiteFox, we adapted its code for compatibility with TVM and ONNXRuntime, following its original paper. To reduce GPT-4 [17] costs in WhiteFox, we substituted it with the open-source Phind-CodeLlama-34B-v2 [3] as the *analysis LLM* and verified its correctness by reproducing its original evaluation results.

The main threat to external validity arises from the choice of DL compilers. We selected TVM and ONNXRuntime, two widely adopted DL compilers. The two DL compilers are particularly representative due to their computational graph formats: Relax IR and ONNX IR, which are compatible with nearly all mainstream DL compilers (as illustrated in Section 4.1.1). Their broad compatibility ensures that the tests generated by OATest can be effectively adopted for testing other DL compilers, underscoring OATest's high generalizability and reducing this threat.

The threat to construct validity mainly arises from the randomness. To reduce this threat, we repeated all experiments involving randomness for five times and reported their aggregated results.

5.2 Optimization-based Fusion Operators

Optimization-based Fusion Operators (OBFOs) are crucial in DL compilers, boosting computational efficiency by reducing redundancy and kernel launch overhead. Since optimizations in DL compilers are context-aware, applying fusion patterns across different contexts can expose new optimization opportunities. Currently, most optimization patterns are handcrafted by experts, yielding

complex fusion algorithms prone to bugs. For example, 5 of 10 confirmed ONNXRuntime bugs detected by OATest occurred in OBFOs. Looking ahead, research in OBFOs should pursue more generalized, automated, and robust approaches. Promising avenues include: (1) Unified ML-driven fusion, which employs learned cost models to generate adaptive fusion strategies, simplifying implementation logic compared to maintaining separate complex algorithms for each optimization, thus reducing bug-prone code; (2) Context-aware fusion mechanisms, which iteratively integrate existing optimization patterns with new operators in the context (like high-order synthesis strategy in OATest) based on real-time system constraints (*e.g.*, memory bandwidth); (3) Formal verification [25], which ensures correctness and mitigates fusion-induced computational errors through SMT-based equivalence and boundary checks.

5.3 Future Work

Current synthesis strategy in OATest utilized random pattern insertion. However, advanced search strategies could enhance the effectiveness of computational graph synthesis. For example, reinforcement learning can be used to guide test generation, where an agent learns to select synthesis points by maximizing the discovery of novel optimization paths in the DL compiler. The state space encodes the current coverage of the optimization path, actions modify the seed graph's synthesis points, and rewards serve as signals to trigger new optimization paths. Integrating such strategies in OATest is a promising future work.

6 Related Work

6.1 DL Compiler Testing

Recently, several techniques have been proposed to test DL compilers [16, 35, 37, 43, 53, 54]. Based on their test generation strategy, they can be primarily categorized into two types: generation-based and mutation-based techniques. The former involves NNSmith [34], HirGen [37], OPERA [43], TVMFuzz [16] and WhiteFox [54], which utilize predefined grammar or LLMs to generate tests from scratch. NNSmith generates computational graphs in the DL model format, emphasizing the diversity of operations and structure. HirGen produces computational graphs in IR format, incorporating diverse data types and shapes to improve diversity. OPERA focuses on generating DL models with a single operator for testing the model loading stage. TVMFuzz focuses on generating hardware-dependent IRs, targeting operator optimizations. WhiteFox, the state-of-the-art technique, leverages LLMs to mine optimization patterns from compiler source code and generate tests. The latter includes MT-DLComp [53] and Tzer [35], which generate tests by modifying existing ones. MT-DLComp [53] applies semantics-preserving mutations to generate equivalent computational graphs for metamorphic testing. Tzer [35] applies joint mutations on hardware-specific IRs and *passes* to explore diverse program states.

Unlike existing testing techniques, we are the first to introduce the computational graph synthesis technique, called OATest, which addresses the challenge of generating optimization-aware computational graphs for testing DL compilers. OATest identifies optimization-aware patterns from documented tests via computational graph analysis and then designs two synthesis strategies to combine patterns with various contexts, producing valid and diverse optimization-aware computational graphs.

6.2 Synthesis-Based Compiler Testing

In traditional compiler testing, synthesis-based techniques [26, 32, 58] have been demonstrated effective in detecting bugs. JavaTailor [58] extracts bug-revealing ingredients from historical bug-triggering tests and combines them into seed programs to generate new tests. LangFuzz [26] leverages bug-revealing code fragments from bug reports to fuzz JavaScript interpreters. Creal [32] extracts C/C++ functions and incorporates them into seed programs by introducing function calls.

Unlike these techniques, OATest aims at synthesizing optimization-aware computational graphs tailored to DL compilers, with distinct test objectives (*i.e.*, DL compiler optimizations) and test formats (*i.e.*, computational graph). OATest generates optimization-aware computational graphs by extracting patterns from documented tests and tailors synthesis strategies to insert the pattern into diverse contexts. Rather than reusing bug-triggering elements from historical tests, OATest emphasizes optimization-aware patterns extraction and synthesis for uncovering optimization bugs in DL compilers. Meanwhile, the test generated by OATest is a computational graph, which fundamentally differs from the test for traditional compilers like JVM [24, 57] and Solidity [38]. This distinction creates unique technical challenges for OATest in generating valid optimization-aware computational graphs, addressed by two novel synthesis strategies: reusing inputs/outputs of existing nodes and creating new connection nodes with compatible inputs/outputs.

7 Conclusion

This paper proposes OATest, a novel optimization-aware technique for computational graph synthesis. First, it extracts optimization-aware patterns from documented tests through code instrumentation. Then, it synthesizes tests by combining extracted patterns with diverse contexts. To ensure the validity and optimization-awareness of synthesized computational graphs, two synthesis strategies are designed, including a strategy to reuse the compatible inputs/outputs of the existing nodes in the context and a strategy to create new nodes with compatible inputs/outputs for connections. Leveraging the implicit test oracles based on crashes and inconsistencies, OATest applies the synthesized computational graphs to testing two popular DL compilers, TVM and ONNXRuntime. The results show that OATest significantly outperforms the state-of-the-art techniques in terms of both bug detection and coverage metrics. By fuzzing the latest versions of TVM and ONNXRuntime for two weeks, OATest identified 56 previously unknown bugs, of which 42/24 have been confirmed/fixed by developers.

Acknowledgments

This work is supported by the National Key Research and Development Program of China (No. 2024YFE0204200), National Natural Science Foundation of China (Grant Nos. 62322208 and 12411530122), and the Hong Kong Research Grant Council/General Research Fund (No. 16206524). Part of this work was done when Qingchao Shen was a visiting student at HKUST under Dr. Yongqiang Tian.

References

[1] Accessed: 2024. Intel OpenVINO. https://docs.openvino.ai/2022.3/home.html.
[2] Accessed: 2024. NVIDIA TensorRT. https://developer.nvidia.com/tensorrt.
[3] Accessed: 2024. Phind-CodeLlama-34B-v2. https://huggingface.co/Phind/Phind-CodeLlama-34B-v2.
[4] Accessed: 2024. PyTorch. https://pytorch.org/.
[5] Accessed: 2024. TVM Issue-17205. https://github.com/apache/tvm/issues/17205.
[6] Accessed: 2024. TVM Issue-17211. https://github.com/apache/tvm/issues/17211.
[7] Accessed: 2024. TVM Issue-17217. https://github.com/apache/tvm/issues/17217.
[8] Accessed: 2024. TVM Issue-17220. https://github.com/apache/tvm/issues/17120.
[9] Accessed: 2024. TVM Issue-17231. https://github.com/apache/tvm/issues/17231.
[10] Accessed: 2024. TVM Issue-17235. https://github.com/apache/tvm/issues/17235.
[11] Accessed: 2024. TVM Issue-17488. https://github.com/apache/tvm/issues/17488.
[12] Accessed: 2024. TVM PR-17218. https://github.com/apache/tvm/pull/17218.
[13] Accessed: 2024. TVM PR-17220. https://github.com/apache/tvm/pull/17220.
[14] Accessed: 2024. TVM PR-17253. https://github.com/apache/tvm/pull/17253.
[15] Accessed: 2024. TVM PR-17501. https://github.com/apache/tvm/pull/17501.
[16] Accessed: 2024. TVMFuzz. https://github.com/dpankratz/TVMFuzz.
[17] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
[18] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2024. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 929–947.
[19] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation* (Carlsbad, CA, USA) *(OSDI'18)*. USENIX Association, USA, 579–594.
[20] Roland Coghetto. 2016. Chebyshev distance. *Formalized Mathematics* 24, 2 (2016), 121–141.
[21] Scott Cyphers, Arjun K Bansal, Anahita Bhiwandiwalla, Jayaram Bobba, Matthew Brookhart, Avijit Chakraborty, Will Constable, Christian Convey, Leona Cook, Omar Kanawi, et al. 2018. Intel ngraph: An intermediate representation, compiler, and executor for deep learning. *arXiv preprint arXiv:1801.08058* (2018).
[22] Vijay D'Silva, Mathias Payer, and Dawn Song. 2015. The correctness-security gap in compiler optimization. In *2015 IEEE Security and Privacy Workshops*. IEEE, 73–87.
[23] Free Software Foundation. 2023. *Gcov: A Test Coverage Program*. GNU Project. https://gcc.gnu.org/onlinedocs/gcc/Gcov.html Accessed: 2025-01-04.
[24] Tianchang Gao, Junjie Chen, Yingquan Zhao, Yuqun Zhang, and Lingming Zhang. 2023. Vectorizing program ingredients for better jvm testing. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 526–537.
[25] Osman Hasan and Sofiene Tahar. 2015. Formal verification methods. In *Encyclopedia of Information Science and Technology, Third Edition*. IGI Global Scientific Publishing, 7162–7170.
[26] Christian Holler, Kim Herzig, and Andreas Zeller. 2012. Fuzzing with Code Fragments. In *21st USENIX Security Symposium (USENIX Security 12)*. USENIX Association, Bellevue, WA, 445–458. https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/holler
[27] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
[28] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 75–86.
[29] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. *ACM Sigplan Notices* 49, 6 (2014), 216–226.
[30] Meiziniu Li, Jialun Cao, Yongqiang Tian, Tsz On Li, Ming Wen, and Shing-Chi Cheung. 2023. COMET: Coverage-Guided Model Generation For Deep Learning Library Testing. *ACM Trans. Softw. Eng. Methodol.* 32, 5, Article 127 (jul 2023), 34 pages. https://doi.org/10.1145/3583566
[31] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2020. The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems* 32, 3 (2020), 708–727.
[32] Shaohua Li, Theodoros Theodoridis, and Zhendong Su. 2024. Boosting Compiler Testing by Injecting Real-World Code. *Proc. ACM Program. Lang.* 8, PLDI, Article 156 (June 2024), 23 pages. https://doi.org/10.1145/3656386
[33] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
[34] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 530–543. https://doi.org/10.1145/3575693.3575707
[35] Jiawei Liu, Yuxiang Wei, Sen Yang, Yinlin Deng, and Lingming Zhang. 2022. Coverage-guided tensor compiler fuzzing with joint ir-pass mutation. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (2022), 1–26.
[36] Haoyang Ma, Alastair F Donaldson, Qingchao Shen, Yongqiang Tian, Junjie Chen, and Shing-Chi Cheung. 2025. Bounded Exhaustive Random Program Generation for Testing Solidity Compilers and Analyzers. *arXiv e-prints* (2025), arXiv–2503.
[37] Haoyang Ma, Qingchao Shen, Yongqiang Tian, Junjie Chen, and Shing-Chi Cheung. 2023. Fuzzing Deep Learning Compilers with HirGen. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 248–260. https://doi.org/10.1145/3597926.3598053
[38] Haoyang Ma, Wuqi Zhang, Qingchao Shen, Yongqiang Tian, Junjie Chen, and Shing-Chi Cheung. 2024. Towards understanding the bugs in solidity compiler. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1312–1324.
[39] ONNXRuntime. 2024. *Graph optimizations in onnx runtime*. https://onnxruntime.ai/docs/performance/model-optimizations/graph-optimizations.html
[40] Linux Test Project. 2023. *Lcov: A Graphical Coverage Tool for GCC*. https://github.com/linux-test-project/lcov Accessed: 2025-01-04.
[41] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Garret Catron, Summer Deng, Roman Dzhabarov, Nick Gibson, James Hegeman, Meghan Lele, Roman Levenstein, et al. 2018. Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907* (2018).
[42] Qingchao Shen, Haoyang Ma, Junjie Chen, Yongqiang Tian, Shing-Chi Cheung, and Xiang Chen. 2021. A comprehensive study of deep learning compiler bugs. In *Proceedings of the 29th ACM Joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 968–980.
[43] Qingchao Shen, Yongqiang Tian, Haoyang Ma, Junjie Chen, Lili Huang, Ruifeng Fu, Shing-Chi Cheung, and Zan Wang. 2024. A Tale of Two DL Cities: When Library Tests Meet Compiler. *arXiv preprint arXiv:2407.16626* (2024).
[44] Toshio Suganuma, Takeshi Ogasawara, Mikio Takeuchi, Toshiaki Yasue, Motohiro Kawahito, Kazuaki Ishizaki, Hideaki Komatsu, and Toshio Nakatani. 2000. Overview of the IBM Java just-in-time compiler. *IBM systems Journal* 39, 1 (2000), 175–193.
[45] Chenyao Suo, Junjie Chen, Shuang Liu, Jiajun Jiang, Yingquan Zhao, and Jianrong Wang. 2024. Fuzzing MLIR compiler infrastructure via operation dependency analysis. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1287–1299.
[46] Chenyao Suo, Jianrong Wang, Yongjia Wang, Jiajun Jiang, Qingchao Shen, and Junjie Chen. 2025. DESIL: Detecting Silent Bugs in MLIR Compiler Infrastructure. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 3093–3119.
[47] MLC team. 2024. *MLC-LLM*. https://github.com/mlc-ai/mlc-llm
[48] TensorFlow. 2024. *An end-to-end platform for machine learning*. https://www.tensorflow.org/
[49] Guancheng Wang, Ruobing Shen, Junjie Chen, Yingfei Xiong, and Lu Zhang. 2021. Probabilistic delta debugging. In *Proceedings of the 29th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 881–892.
[50] Haoyu Wang, Junjie Chen, Chuyue Xie, Shuang Liu, Zan Wang, Qingchao Shen, and Yingquan Zhao. 2023. Mlirsmith: Random program generation for fuzzing mlir compiler infrastructure. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1555–1566.
[51] Zihan Wang, Pengbo Nie, Xinyuan Miao, Yuting Chen, Chengcheng Wan, Lei Bu, and Jianjun Zhao. 2023. GenCoG: A DSL-Based Approach to Generating Computation Graphs for TVM Testing. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 904–916.
[52] Mingyuan Wu, Minghai Lu, Heming Cui, Junjie Chen, Yuqun Zhang, and Lingming Zhang. 2023. JITfuzz: Coverage-guided Fuzzing for JVM Just-in-Time Compilers. In 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). 56–68.
[53] Dongwei Xiao, Zhibo Liu, Yuanyuan Yuan, Qi Pang, and Shuai Wang. 2022. Metamorphic testing of deep learning compilers. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 1 (2022), 1–28.
[54] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2023. White-box compiler fuzzing empowered by large language models. *arXiv preprint arXiv:2310.15991* (2023).
[55] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 283–294. https://doi.org/10.1145/1993498.1993532
[56] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on software engineering* 28, 2 (2002), 183–200.

[57] Yingquan Zhao, Zan Wang, Junjie Chen, Ruifeng Fu, Yanzhou Lu, Tianchang Gao, and Haojie Ye. 2024. Program Ingredients Abstraction and Instantiation for Synthesis-based JVM Testing. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3943–3957.

[58] Yingquan Zhao, Zan Wang, Junjie Chen, Mengdi Liu, Mingyuan Wu, Yuqun Zhang, and Lingming Zhang. 2022. History-driven test program synthesis for JVM testing. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1133–1144. https://doi.org/10.1145/3510003.3510059

[59] Chijin Zhou, Bingzhou Qian, Gwihwan Go, Quan Zhang, Shanshan Li, and Yu Jiang. 2024. PolyJuice: Detecting Mis-compilation Bugs in Tensor Compilers with Equality Saturation Based Rewriting. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 1309–1335.